

Data Parallelism Implementation

DDP stands for Distributed Data Parallel, a training wrapper that replicates a model on multiple devices and averages gradients across them via collectives like all-reduce. After synchronization, each replica applies the same optimizer step so all copies stay in lockstep for correct, scalable training.

Steps

1. Deterministic, non-overlapping data sharding per rank
2. Forward + backward per device independently
3. Collective gradient synchronization (all-reduce)
4. Identical parameter updates on all devices

Good practices

- Use a seeded DistributedSampler for dataloaders
- Overlap communication and compute via gradient bucketing
- Configure DDP for unused params when models branch (find_unused_parameters)
- Gradient clipping before the optimizer step
- Mixed precision: prefer BF16 if available; otherwise FP16 with dynamic loss scaling

Notes on synchronization

- Ring all-reduce minimizes bandwidth and scales with $N-1$ hops
- Ensure optimizer steps are identical across ranks after sync

Scalability Analysis

Sources of scaling loss

- Communication overhead: bandwidth limits, latency, protocol inefficiency
- Synchronization barriers and stragglers: everyone waits for the slowest
- Load imbalance/heterogeneity: uneven work, throttling, ECC, I/O skew
- Memory pressure: activations/optimizer state constrain batch/parallelism

Measuring efficiency

- Ideal speedup = N
- Actual speedup = T_1 / T_N
- Parallel efficiency = $(T_1 / T_N) / N$

Amdahl's Law (upper bound on speedup)

- Speedup $\leq 1 / (s + (1 - s)/N)$, where s = serial fraction
- Example: $s = 0.10 \Rightarrow \text{max speedup} = 10\times \text{ as } N \rightarrow \infty$
- Even small s severely caps efficiency at large N

What to do (reduce s , hide comm)

- Overlap communication with compute: gradient bucketing, fused kernels
- Hierarchical collectives; topology-aware placement
- Lower-precision or compressed gradients with error feedback
- Algorithmic options: increase local batch; partial/delayed sync; asynchronous variants with care
- Balance work: uniform microbatching, stage rebalancing, dynamic batch assignment
- Detect and mitigate stragglers: health checks, retry, prefetch, pinned workers

Distributed Systems Concerns in Practice

Fabrics and topology:

- Intra-node: PCIe, NVLink
- Inter-node: 25/100/200/400 Gbps Ethernet/InfiniBand; latencies 2–10× higher

Collectives vs parameter server:

- Collectives (NCCL/Gloo/MPI): bandwidth-efficient, symmetric
- PS: flexible, supports async, but can bottleneck at servers

Orchestration & launch:

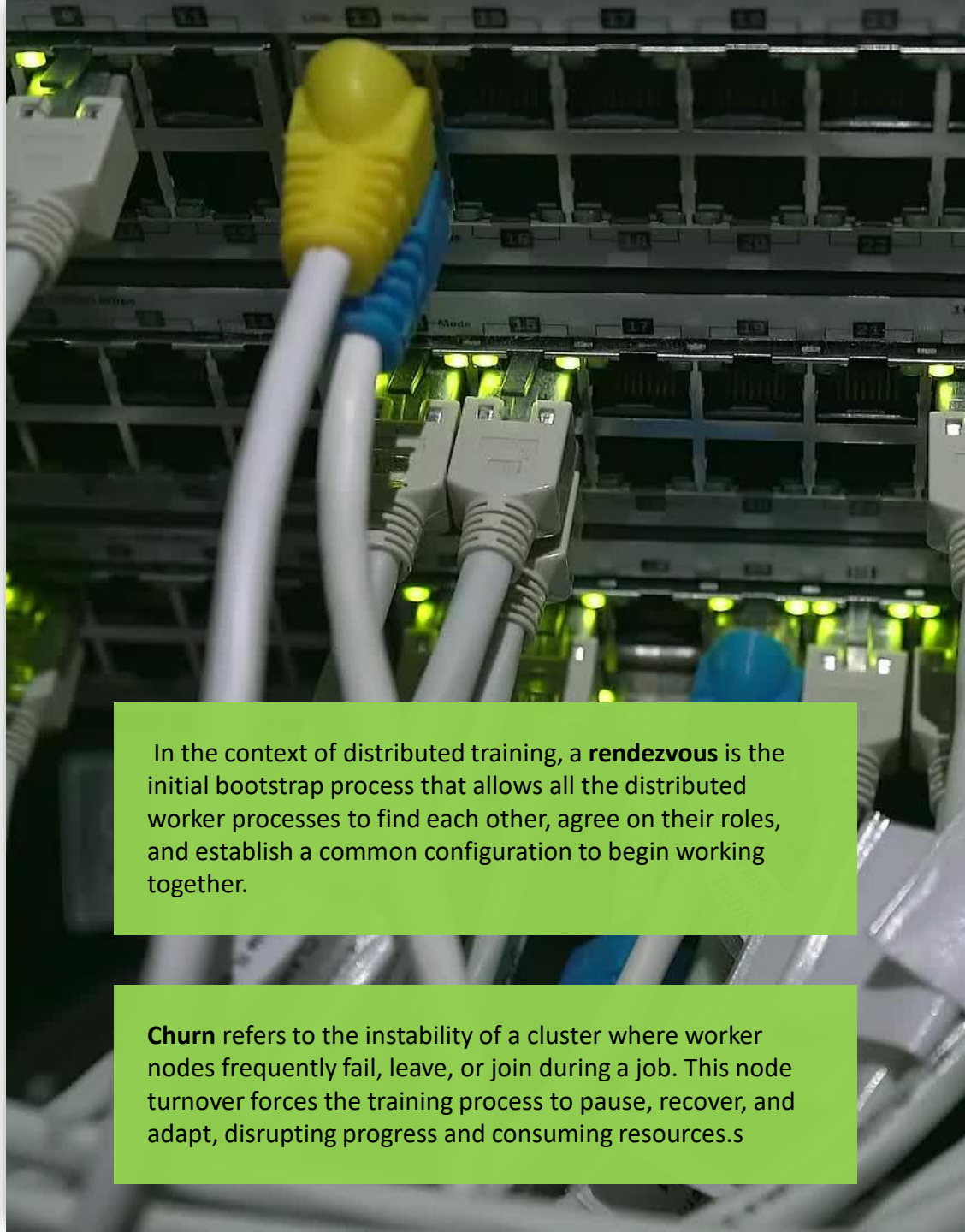
- SLURM/Kubernetes; ranks, world size, rendezvous; topology-aware placement

Reproducibility:

- Deterministic sharding and seeding; avoid nondeterministic kernels

Elasticity:

- Handle node churn; reshard data and adjust LR safely



In the context of distributed training, a **rendezvous** is the initial bootstrap process that allows all the distributed worker processes to find each other, agree on their roles, and establish a common configuration to begin working together.

Churn refers to the instability of a cluster where worker nodes frequently fail, leave, or join during a job. This node turnover forces the training process to pause, recover, and adapt, disrupting progress and consuming resources.

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable

Topology-Aware & Hierarchical All-Reduce; Libraries

- Libraries:
 - NCCL (GPU-centric), Gloo (CPU/GPU), MPI (HPC)
- Algorithms:
 - Ring: bandwidth-optimal for large messages
 - Tree: lower latency for smaller messages
 - Reduce-scatter + all-gather: core of sharded training
- Hierarchical all-reduce:
 - Intra-node reduce-scatter (NVLink)
 - Inter-node reduce-scatter (IB/Ethernet)
 - Inter-node all-gather
 - Intra-node all-gather
- Practice: bucket sizes, algorithm selection per tensor size, topology hints

Communication- Efficient Protocols

Core idea: cut bytes per round without hurting accuracy

Techniques

- **Quantization:** send updates in 8/4/2 bits with per-tensor scales
- **Sparsification:** top-k or thresholded updates; error feedback to recover dropped mass
- **Structured updates:** layer-wise or last-layer only; low-rank adapters (LoRA/PEFT)
- **Delta/periodic sync:** send changes since last round; skip small updates

Practicalities

- Clip and add DP noise if required; secure aggregation friendly
- Calibrate k/bit-width; monitor loss drift and add error feedback
- Compress after aggregation when possible to reduce server load

Fault Tolerance and Reliability

Participation churn

- Over-invite; accept first N updates; retry/backoff; resumable uploads

Asynchronous/partial aggregation

- Quorum-based rounds; rolling aggregation; staleness-aware weighting



Staleness management

- Tag updates with model version; decay weights by lag; bound max staleness

Robustness to bad/malicious clients

- Norm clipping, validation-on-server, robust aggregators (median/trimmed mean)
- Outlier and consistency checks; reputation/blacklisting

Resilience engineering

- Idempotent jobs, checkpointing, rate limiting, circuit breakers
- Telemetry/alerts for dropout spikes and drift

A **quorum** is the minimum number of clients that must respond before the server proceeds with a training round. Instead of waiting for every single client, which could take forever, it aggregates updates once this representative group has reported in.